

FRED TALKS

24th July 2026

CONTENTS

Eight years of wanting, three months of building with AI

LALIT MAGANTI · 3910 WORDS

Building agents that reach production systems with MCP

CLAUDE.COM · 1746 WORDS

Search has its own bitter lesson

DOUG TURNBULL · 1012 WORDS

Eight years of wanting, three months of building with AI

LALIT MAGANTI · 12 APR 2026 · [SOURCE](#)

For eight years, I've wanted a high-quality set of devtools for working with SQLite. Given how important SQLite is to the industry¹, I've long been puzzled that no one has invested in building a really good developer experience for it².

A couple of weeks ago, after ~250 hours of effort over three months³ on evenings, weekends, and vacation days, I finally [released syntaqlite \(GitHub\)](#), fulfilling this long-held wish. And I believe the main reason this happened was because of AI coding agents⁴.

Of course, there's no shortage of posts claiming that AI one-shot their project or pushing back and declaring that AI is all slop. I'm going to take a very different approach and, instead, systematically break down my experience building syntaqlite with AI, both where it helped *and* where it was detrimental.

I'll do this while contextualizing the project and my background so you can independently assess how generalizable this experience was. And whenever I make a claim, I'll try to back it up with evidence from my project journal, coding transcripts, or commit history⁵.

In my work on [Perfetto](#), I maintain a SQLite-based language for querying performance traces called [PerfettoSQL](#). It's basically the same as SQLite but with a few extensions to make the trace querying experience better. There are ~100K lines of PerfettoSQL internally in Google and it's used by a wide range of teams.

Having a language which gets traction means your users also start expecting things like formatters, linters, and editor extensions. I'd hoped that we could adapt some SQLite tools from open source but the more I looked into it, the more disappointed I was. What I found either wasn't reliable enough, fast enough⁶, or flexible enough to adapt to PerfettoSQL. There was clearly an opportunity to build something from scratch, but it was never the "most important thing we could work on". We've been reluctantly making do with the tools out there but always wishing for better.

On the other hand, there *was* the option to do something in my spare time. I had built lots of open source projects in my teens⁷ but this had faded away during university when I felt that I just didn't have the motivation anymore. Being a maintainer is much more than just "throwing the code out there" and seeing what happens. It's triaging bugs, investigating crashes, writing documentation, building a community, and, most importantly, having a direction for the project.

But the itch of open source (specifically freedom to work on what I wanted while helping others) had never gone away. The SQLite devtools project was eternally in my mind as "something I'd like to work on". But there was another reason why I kept putting it off: it sits at the intersection of being both hard *and* tedious.

What makes it hard and tedious

If I was going to invest my personal time working on this project, I didn't want to build something that only helped Peretto: I wanted to make it work for *any* SQLite user out there⁸. And this means parsing SQL *exactly* like SQLite.

The heart of any language-oriented devtool is the parser. This is responsible for turning the source code into a "parse tree" which acts as the central data structure anything else is built on top of. If your parser isn't accurate, then your formatters and linters will inevitably inherit those inaccuracies; many of the tools I found suffered from having parsers which approximated the SQLite language rather than representing it precisely.

Unfortunately, unlike many other languages, SQLite has no formal specification describing how it should be parsed. It doesn't expose a stable API for its parser either. In fact, quite uniquely, in its implementation it doesn't even build a parse tree at all⁹! The only reasonable approach left in my opinion is to carefully extract the relevant parts of SQLite's source code and adapt it to build the parser I wanted¹⁰.

This means getting into the weeds of SQLite source code, a fiendishly difficult codebase to understand. The whole project is written in C in an incredibly dense style; I've spent days just understanding the virtual table API¹¹ and implementation. Trying to grasp the full parser stack was daunting.

There's also the fact that there are >400 rules in SQLite which capture the full surface area of its language. I'd have to specify in each of these "grammar rules" how that part of the syntax maps to the matching node in the parse tree. It's extremely repetitive work; each rule is similar to all the ones around it but also, by definition, different.

And it's not just the rules but also coming up with and writing tests to make sure it's correct, debugging if something is wrong, triaging and fixing the inevitable bugs people filed when I got something wrong...

For years, this was where the idea died. Too hard for a side project¹², too tedious to sustain motivation, too risky to invest months into something that might not work.

I've been using coding agents since early 2025 (Aider, Roo Code, then Claude Code since July) and they'd definitely been useful but never something I felt I could trust a serious project to. But towards the end of 2025, the models seemed to make a significant step forward in quality¹³. At the same time, I kept hitting problems in Perfetto which would have been trivially solved by having a reliable parser. Each workaround left the same thought in the back of my mind: maybe it's finally time to build it for real.

I got some space to think and reflect over Christmas and decided to really stress test the most maximalist version of AI: could I vibe-code the whole thing using just Claude Code on the Max plan (£200/month)?

Through most of January, I iterated, acting as semi-technical manager and delegating almost all the design and all the implementation to Claude. Functionally, I ended up in a reasonable place: a parser in C extracted from SQLite sources using a bunch of Python scripts, a formatter built on top, support for both the SQLite language and the PerfettoSQL extensions, all exposed in a web playground.

But when I reviewed the codebase in detail in late January, the downside was obvious: the codebase was complete spaghetti¹⁴. I didn't understand large parts of the Python source extraction pipeline, functions were scattered in random files without a clear shape, and a few files had grown to several thousand lines. It was *extremely* fragile; it solved the immediate problem *but* it was never going to cope with my larger vision, never mind integrating it into the Perfetto tools. The saving grace was that it had proved the approach was viable and generated more than 500 tests, many of which I felt I could reuse.

I decided to throw away everything and start from scratch while also switching most of the codebase to Rust¹⁵. I could see that C was going to make it difficult to build the higher level components like the validator and the language server implementation. And as a bonus, it would also let me use the same language for both the extraction and runtime instead of splitting it across C and Python.

More importantly, I completely changed my role in the project. I took ownership of all decisions¹⁶ and used it more as “autocomplete on steroids” inside a much tighter process: opinionated design upfront, reviewing every change thoroughly, fixing problems eagerly as I spotted them, and investing in scaffolding (like linting, validation, and non-trivial testing¹⁷) to check AI output automatically.

The core features came together through February and the final stretch (upstream test validation, editor extensions, packaging, docs) led to a 0.1 launch in mid-March.

But in my opinion, this timeline is the least interesting part of this story. What I really want to talk about is what wouldn’t have happened without AI and also the toll it took on me as I used it.

AI is why this project exists, and why it’s as complete as it is

I’ve written in the past about how one of my biggest weaknesses as a software engineer is my tendency to procrastinate when facing a big new project. Though I didn’t realize it at the time, it could not have applied more perfectly to building syntaqlite.

AI basically let me put aside all my doubts on technical calls, my uncertainty of building the right thing and my reluctance to get started by giving me very concrete problems to work on. Instead of “I need to understand how SQLite’s parsing works”, it was “I need to get AI to suggest an approach for me so I can tear it up and build something better”¹⁸. I work so much better with concrete prototypes to play with and code to look at than endlessly thinking about designs in my head, and AI lets me get to that point at a pace I could not have dreamed about before. Once I took the first step, every step after that was so much easier.

AI turned out to be better than me at the act of writing code itself, assuming that code is obvious. If I can break a problem down to “write a function with this behaviour and parameters” or “write a class matching this interface,” AI will build it faster than I would and, crucially, in a style that might well be more intuitive to a future reader. It documents things I’d skip, lays out code consistently with the rest of the project, and sticks to what you might call the “standard dialect” of whatever language you’re working in¹⁹.

That standardness is a double-edged sword. For the vast majority of code in any project, standard is exactly what you want: predictable, readable, unsurprising. But every project has pieces that are its edge, the parts where the value comes from doing something non-obvious. For

syntaqlite, that was the extraction pipeline and the parser architecture. AI's instinct to normalize was actively harmful there, and those were the parts I had to design in depth and often resorted to just writing myself.

But here's the flip side: the same speed that makes AI great at obvious code also makes it great at refactoring. If you're using AI to generate code at industrial scale, you *have* to refactor constantly and continuously²⁰. If you don't, things immediately get out of hand. This was the central lesson of the vibe-coding month: I didn't refactor enough, the codebase became something I couldn't reason about, and I had to throw it all away. In the rewrite, refactoring became the core of my workflow. After every large batch of generated code, I'd step back and ask "is this ugly?" Sometimes AI could clean it up. Other times there was a large-scale abstraction that AI couldn't see but I could; I'd give it the direction and let it execute²¹. If you have taste, the cost of a wrong approach drops dramatically because you can restructure quickly²².

Of all the ways I used AI, research had by far the highest ratio of value delivered to time spent.

I've worked with interpreters and parsers before but I had never heard of Wadler-Lindig pretty printing²³. When I needed to build the formatter, AI gave me a concrete and actionable lesson from a point of view I could understand and pointed me to the papers to learn more. I could have found this myself eventually, but AI compressed what might have been a day or two of reading into a focused conversation where I could ask "but why does this work?" until I actually got it.

This extended to entire domains I'd never worked in. I have deep C++ and Android performance expertise but had barely touched Rust tooling or editor extension APIs. With AI, it wasn't a problem: the fundamentals are the same, the terminology is similar, and AI bridges the gap²⁴. The VS Code extension would have taken me a day or two of learning the API before I could even start. With AI, I had a working extension within an hour.

It was also invaluable for reacquainting myself with parts of the project I hadn't looked at for a few days²⁵. I could control how deep to go: "tell me about this component" for a surface-level refresher, "give me a detailed linear walkthrough" for a deeper dive, "audit unsafe usages in this repo" to go hunting for problems. When you're context switching a lot, you lose context fast. AI let me reacquire it on demand.

Beyond making the project exist at all, AI is also the reason it shipped as complete as it did. Every open source project has a long tail of features that are important but not critical: the things you know theoretically how to do but keep deprioritizing because the core work is more

pressing. For syntaqlite, that list was long: editor extensions, Python bindings, a WASM playground, a docs site, packaging for multiple ecosystems²⁶. AI made these cheap enough that skipping them felt like the wrong trade-off.

It also freed up mental energy for UX²⁷. Instead of spending all my time on implementation, I could think about what a user's first experience should feel like: what error messages would actually help them fix their SQL, how the formatter output should look by default, whether the CLI flags were intuitive. These are the things that separate a tool people try once from one they keep using, and AI gave me the headroom to care about them. Without AI, I would have built something much smaller, probably no editor extensions or docs site. AI didn't just make the same project faster. It changed what the project was.

There's an uncomfortable parallel between using AI coding tools and playing slot machines²⁸. You send a prompt, wait, and either get something great or something useless. I found myself up late at night wanting to do "just one more prompt," constantly trying AI just to see what would happen even when I knew it probably wouldn't work. The sunk cost fallacy kicked in too: I'd keep at it even in tasks it was clearly ill-suited for, telling myself "maybe if I phrase it differently this time."

The tiredness feedback loop made it worse²⁹. When I had energy, I could write precise, well-scoped prompts and be genuinely productive. But when I was tired, my prompts became vague, the output got worse, and I'd try again, getting more tired in the process. In these cases, AI was probably slower than just implementing something myself, but it was too hard to break out of the loop³⁰.

Several times during the project, I lost my mental model of the codebase³¹. Not the overall architecture or how things fitted together. But the day-to-day details of what lived where, which functions called which, the small decisions that accumulate into a working system. When that happened, surprising issues would appear and I'd find myself at a total loss to understand what was going wrong. I hated that feeling.

The deeper problem was that losing touch created a communication breakdown³². When you don't have the mental thread of what's going on, it becomes impossible to communicate meaningfully with the agent. Every exchange gets longer and more verbose. Instead of "change FooClass to do X," you end up saying "change the thing which does Bar to do X". Then the agent has to figure out what Bar is, how that maps to FooClass, and sometimes it gets it wrong³³. It's exactly the same complaint engineers have always had about managers who don't understand the code asking for fanciful or impossible things. Except now you've become that manager.

The fix was deliberate: I made it a habit to read through the code immediately after it was implemented and actively engage to see “how would I have done this differently?”.

Of course, in some sense all of the above is also true of code I wrote a few months ago (hence the sentiment that AI code is legacy code), but AI makes the drift happen faster because you’re not building the same muscle memory that comes from originally typing it out.

There were some other problems I only discovered incrementally over the three months.

I found that AI made me procrastinate on key design decisions³⁴. Because refactoring was cheap, I could always say “I’ll deal with this later.” And because AI could refactor at the same industrial scale it generated code, the cost of deferring felt low. But it wasn’t: deferring decisions corroded my ability to think clearly because the codebase stayed confusing in the meantime. The vibe-coding month was the most extreme version of this. Yes, I understood the problem, but if I had been more disciplined about making hard design calls earlier, I could have converged on the right architecture much faster.

Tests created a similar false comfort³⁵. Having 500+ tests felt reassuring, and AI made it easy to generate more. But neither humans nor AI are creative enough to foresee every edge case you’ll hit in the future; there are several times in the vibe-coding phase where I’d come up with a test case and realise the design of some component was completely wrong and needed to be totally reworked. This was a significant contributor to my lack of trust and the decision to scrap everything and start from scratch.

Basically, I learned that the “normal rules” of software still apply in the AI age: if you don’t have a fundamental foundation (clear architecture, well-defined boundaries) you’ll be left eternally chasing bugs as they appear.

Something I kept coming back to was how little AI understood about the passage of time³⁶. It sees a codebase in a certain state but doesn’t *feel* time the way humans do. I can tell you what it feels like to use an API, how it evolved over months or years, why certain decisions were made and later reversed.

The natural problem from this lack of understanding is that you either make the same mistakes you made in the past and have to relearn the lessons *or* you fall into new traps which were successfully avoided the first time, slowing you down in the long run. In my opinion, this is a similar problem to why losing a high-quality senior engineer hurts a team so much: they carry history and context that doesn’t exist anywhere else and act as a guide for others around them.

In theory, you can try to preserve this context by keeping specs and docs up to date. But there's a reason we didn't do this before AI: capturing implicit design decisions exhaustively is incredibly expensive and time-consuming to write down. AI can help draft these docs, but because there's no way to automatically verify that it accurately captured what matters, a human still has to manually audit the result. And that's still time-consuming.

There's also the context pollution problem. You never know when a design note about API A will echo in API B. Consistency is a huge part of what makes codebases work, and for that you don't just need context about what you're working on right now but also about other things which were designed in a similar way. Deciding what's relevant requires exactly the kind of judgement that institutional knowledge provides in the first place.

Reflecting on the above, the pattern of when AI helped and when it hurt was fairly consistent.

When I was working on something I already understood deeply, AI was excellent. I could review its output instantly, catch mistakes before they landed and move at a pace I'd never have managed alone. The parser rule generation is the clearest example³⁷: I knew exactly what each rule should produce, so I could review AI's output within a minute or two and iterate fast.

When I was working on something I could describe but didn't yet know, AI was good but required more care. Learning Wadler-Lindig for the formatter was like this: I could articulate what I wanted, evaluate whether the output was heading in the right direction, and learn from what AI explained. But I had to stay engaged and couldn't just accept what it gave me.

When I was working on something where I didn't even know what I wanted, AI was somewhere between unhelpful and harmful. The architecture of the project was the clearest case: I spent weeks in the early days following AI down dead ends, exploring designs that felt productive in the moment but collapsed under scrutiny. In hindsight, I have to wonder if it would have been faster just thinking it through without AI in the loop at all.

But expertise alone isn't enough. Even when I understood a problem deeply, AI still struggled if the task had no objectively checkable answer³⁸. Implementation has a right answer, at least at a local level: the code compiles, the tests pass, the output matches what you asked for. Design doesn't. We're still arguing about OOP decades after it first took off.

Concretely, I found that designing the public API of `syntaqlite` was where this hit home the hardest. I spent several days in early March doing nothing but API refactoring, manually fixing things any experienced engineer would have instinctively avoided but AI made a total mess of. There's no test or objective metric for "is this API pleasant to use" and "will this API help users solve the problems they have" and that's exactly why the coding agents did *so badly* at it.

This takes me back to the days I was obsessed with physics and, specifically, relativity. The laws of physics look simple and Newtonian in any small local area, but zoom out and spacetime curves in ways you can't predict from the local picture alone. Code is the same: at the level of a function or a class, there's usually a clear right answer, and AI is excellent there. But architecture is what happens when all those local pieces interact, and you can't get good global behaviour by stitching together locally correct components.

Knowing where you are on these axes at any given moment is, I think, the core skill of working with AI effectively.

Eight years is a long time to carry a project in your head. Seeing these SQLite tools actually exist and function after only three months of work is a massive win, and I'm fully aware they wouldn't be here without AI.

But the process wasn't the clean, linear success story people usually post. I lost an entire month to vibe-coding. I fell into the trap of managing a codebase I didn't actually understand, and I paid for that with a total rewrite.

The takeaway for me is simple: AI is an incredible force multiplier for implementation, but it's a dangerous substitute for design. It's brilliant at giving you the right answer to a specific technical question, but it has no sense of history, taste, or how a human will actually feel using your API. If you rely on it for the "soul" of your software, you'll just end up hitting a wall faster than you ever have before.

What I'd like to see more of from others is exactly what I've tried to do here: honest, detailed accounts of building real software with these tools; not weekend toys or one-off scripts but the kind of software that has to survive contact with users, bug reports, and your own changing mind.

Building agents that reach production systems with MCP

CLAUDE.COM · 23 APR 2026 · [SOURCE](#)

Agents are only as useful as the systems they can reach. Teams tend to converge on three approaches for connecting them to external systems—direct API calls, CLIs, and MCP. This post lays out where each fits, why production agents tend to land on MCP, and the patterns for building those integrations effectively.

Connecting agents to external systems

We generally see three paths for connecting agents to external systems: direct API calls, CLIs, and MCP. Each makes sense somewhere, depending on what you're building. The key distinction is whether there's a common layer between agents and services, and how far that layer reaches.

Direct API calls

The agent calls your API directly—either by writing code that issues HTTP requests inside a code-execution sandbox, or through a generic function-calling tool. This is where most teams start, and it works fine for one agent talking to one service, or a small number of integrations that don't need to be reused across agent platforms.

The challenges start to hit at scale. With no common layer between agents and services, each agent–service pair becomes a bespoke integration with its own auth handling, tool descriptions, and edge cases—the $M \times N$ integration problem.

Command-line interface (CLI)

The agent runs your command-line tool in a shell. This is fast, lightweight, and leans on pre-existing tooling. It works great for local environments and sandboxed containers—anywhere there's a filesystem and a shell. This provides a common layer, but it's thin.

CLIs hit hard limits reaching mobile, web, or cloud-hosted platforms that don't expose a container, and auth is handled by the CLI's own mechanism—usually a credential file on disk. This is best suited to quick, permissive integrations in local environments.

Model Context Protocol (MCP)

MCP provides the common layer as a protocol. The agent connects to a server that exposes your system's capabilities, with auth, discovery, and rich semantics standardized. One remote server reaches any compatible client (Claude, ChatGPT, Cursor, VS Code, and more), in any deployment environment.

It requires a little bit more upfront investment. The return is that the integration is portable, and provides the semantics needed for a feature-rich agent integration.

Production agents run in the cloud

Production agents increasingly run in the cloud, so they can scale and operate continuously. The systems they need to reach are cloud-hosted too: where your data lives, work is tracked, and your infrastructure runs. Often these systems are remote and behind auth, where MCP provides the common layer.

We're already seeing this in adoption. The [MCP SDKs](#) recently surpassed 300 million downloads a month, up from 100 million at the start of the year, with strong adoption across enterprises and popular agentic platforms. Millions of people use MCP with Claude every day, and the protocol underpins much of what we've shipped recently, including [Claude Cowork](#), [Claude Managed Agents](#), and [channels in Claude Code](#).

As MCP continues to support production agentic systems, we're sharing patterns for building these integrations well: from building advanced servers to context-efficient clients, and where skills complement the protocol.

Building effective MCP servers

We have over 200 MCP servers in our [directory](#), used by millions of people every day. From working closely with enterprises and developers building on the protocol, we've spotted a handful of design patterns that determine how reliably agents can use a server.

Build remote servers for maximum reach

A remote server is what gives you distribution—it's the only configuration that runs across web, mobile, and cloud-hosted agents, and it's what every major client is optimized to consume. Build remote servers so agents can use your system wherever they run.

Group tools around intent, not endpoints

Fewer, well-described tools consistently outperform exhaustive API mirrors. Don't wrap your API into an MCP server one-to-one—group tools around intent, so the agent can accomplish a task in a couple of calls instead of stitching many primitives together. A single `create_issue_from_thread` tool beats `get_thread` + `parse_messages` + `create_issue` + `link_attachment`. See [writing effective tools for agents](#) to learn more about the full pattern.

Design for code orchestration when your surface is large

If your service requires hundreds of distinct operations, such as Cloudflare, AWS, or Kubernetes, an intent-grouped toolset likely won't cover it. Instead, expose a thin tool surface that accepts code: the agent writes a short script, your server runs it in a sandbox against your API, and only the result returns. [Cloudflare's MCP server](#) is the reference example—two tools (search and execute) cover ~2,500 endpoints in roughly 1K tokens.

Ship rich semantics where they help

[MCP Apps](#) is the first official protocol extension and lets a tool return an interactive interface, such as a chart, form, or dashboard, all rendered inline in the chat interface. Servers that ship MCP apps tend to see meaningfully higher adoption and retention than those that return text alone. Use it to put your product's UI in front of agents or end-users at the moment it matters—the extension is supported in Claude.ai, Claude Cowork, and many other top AI tools.

[Elicitation](#) lets your server pause mid-tool call to ask the user for input. [Form mode](#) sends a simple schema and the client renders a native form—use it to request a missing parameter, confirm a destructive action, or disambiguate options. [URL mode](#) hands the user to a browser—use it to complete downstream OAuth, take a payment, or collect any credential that should never transit the MCP client. Both keep the user in the flow instead of sending them to a settings page. Form mode is supported broadly; URL mode is supported in Claude Code, with more clients in progress.

Lean on standardized auth

Standardized auth makes MCP practical for cloud-hosted agents. If your server requires OAuth, the latest [MCP spec](#) supports [CIMD](#) (Client ID Metadata Documents) for client registration—it gives users a fast first-time auth flow and far fewer surprise re-auth prompts. This is our recommended approach for auth, the capability is supported in MCP SDKs, Claude.ai, and Claude Code, and is being broadly adopted across the industry.

Once a user has authorized, the next question is how a cloud-hosted agent holds and reuses those tokens at runtime. [Vaults](#) in [Claude Managed Agents](#) covers this: register a user's OAuth tokens once, reference the vault by ID at session creation, and the platform injects the right credentials into each MCP connection and refreshes them on your behalf—no secret store to build, no tokens to pass around per call.

Making MCP clients more context-efficient

MCP standardizes how AI agents (*clients*) connect to and work with tools and data sources they need (*servers*). The server securely exposes a range of capabilities, while the client orchestrates them and manages context. If you're building an MCP client, make it context-efficient with patterns for progressive disclosure.

Load tool definitions on demand with tool search

Tool search defers loading all tools into context, rather than loading them upfront. This allows the agent to search the catalog at runtime, pulling in the relevant tools when needed. In our testing, tool search tends to cut tool-definition tokens by 85%+ while maintaining high selection accuracy.

Reducing context usage with tool search. Source: advanced tool use

Process tool results in code with programmatic tool calling

Programmatic tool calling processes tool results in a code-execution sandbox, rather than returning them raw to the model. This lets the agent loop, filter, and aggregate across calls in code, with only the final output reaching context. In our testing, this reduces token usage by roughly 37% on complex multi-step workflows.

Together, these patterns compose naturally across multiple servers: leaner context, fewer round-trips, faster responses. See advanced tool use for the full breakdown.

Pairing MCP servers with skills

Skills and MCP are complementary. MCP gives an agent access to tools and data from external systems, while skills teach an agent the procedural knowledge of *how* to use those tools to accomplish real work. The most capable agents use both, and skills make MCP servers scale beyond a handful of connections. There are two general patterns for combining them:

Bundle skills and MCP servers as a plugin

Plugins for Claude are a useful abstraction that allow developers to bundle skills, MCP servers, hooks, LSP servers, and specialized subagents in one easily-consumable distribution method. Using this approach is the best way to unify multiple context providers with minimal friction.

Combining MCP servers with skills allows Claude to act more like a domain-specialist. Grab your tools via MCP, and give Claude the skills to orchestrate workflows end-to-end. See our [data plugin](#) for Cowork as an example, which consists of 10 skills and 8 MCP servers for apps like Snowflake, Databricks, BigQuery, Hex and more.

Combining skills with MCP. Source: [Extending Claude's capabilities with skills and MCP servers](#)

Distribute skills from an MCP server

It's increasingly common for providers to publish a skill alongside their MCP server, so the agent gets both the raw capabilities and an opinionated playbook for using them well. [Canva](#), [Notion](#), [Sentry](#), and more do this today in Claude, listing the skill next to their connector in our [web directory](#).

To make that pairing portable across every client, the MCP community is actively working on an [extension](#) for delivering skills directly from servers. This way the client inherits the relevant expertise automatically, versioned with the API it depends on. We expect this pattern to see broad adoption as the extension stabilizes.

The compounding layer

We opened with three paths for connecting agents to external systems. In practice, mature integrations will ship all three: the API as the foundation, a CLI for local-first environments, and MCP for cloud-based agents.

As production agents move to the cloud, MCP becomes the critical layer, and it's the one that compounds. Today, a remote server reaches every compatible client across any deployment environment, with auth, interactivity, and rich semantics handled by the protocol. As more clients adopt the spec and more extensions land in it, that same server gets more capable without you shipping anything new.

When building an integration, if your goal is to have production agents in the cloud reach your system, build an MCP server and make it excellent using the patterns above. Every integration built on MCP strengthens the ecosystem: fewer edge cases to solve alone, fewer bespoke integrations to maintain.

Acknowledgements

Thanks to Den Delimarsky, David Soria Parra, Henry Shi, Felix Rieseberg, Conor Kelly, Molly Vorwerck, Andy Schumeister, Kevin Garcia, Amie Rotherham, Matt Samuels, Angela Jiang, Katelyn Lesse, AJ Rebeiro and Jess Yan for their contributions to this blog.

Search has its own bitter lesson

DOUG TURNBULL · 22 JUN 2026 · [SOURCE](#)

An agent with grep does quite well at search.

It's shocking to search technologists. We want to cocoon the problem in technology. We care about algorithms, knowledge graphs, ranking: ever-and-ever smarter retrieval.

It turns out, though, you can play a bit of a trick. If you convince everyone to optimize content for your search engine, you'll have built the best search engine.

There's Sutton's [bitter lesson](#) about unleashing raw compute on a problem. But there's a different bitter lesson in search: algorithms matter less than convincing the world to optimize for your search engine.

Sure, technology acts as the trigger. In Web Search, Google shifted away from text relevance to prioritize authoritativeness (PageRank). Their success told the market: create content others want to link to. They reshaped the Web for good (or ill) around that incentive.

Google crowdsourced search quality to the Web. Claude Code outsources it to every developer.

Developers depend on agents to find documentation on the file system. We like that dopamine hit. That thrill of Claude Code knowing exactly how to reason about our code.

When agents make boneheaded mistakes, we stress. We frantically update documentation, we link to it from AGENTS.md, we add a new document, we place it in a useful location. We do anything - absolutely anything - to ensure agents know how to find and understand documentation.

Just as the SEO expert reads the tea leaves of Google's algorithm. The software developer writes to the LLM to make documentation findable + useful.

A search system with these incentives overcomes technologically sophisticated systems. For example, classic RAG rests on a foundation of technically sophisticated question answering. Answers often look like conversationally valid responses, but with little linkage back to questions it might answer:

Question:

| What is a runner test?

Answer:

| Verifies that small-dollar deposits and withdrawals can be executed against real test bank accounts during account reconciliation.

Yes that's literally an answer to the question.

But I wouldn't consider this "good documentation". It's not anchored to any concepts from the question. It's just an out of context bit of text.

If you wrote this for a coding agent, you'd probably organize the chunk to give hints about why it's useful.

| docs/tests/runner.md

| # Runner Tests

| ## Purpose

| We need to ensure the system runs end to end

| ## Usage

| Use this style of test sparingly. Unit and standard e2e tests should usually be preferred.

| ## What is it

| A runner test actually interacts with real-life test bank accounts to ensure we can correctly deposit / withdraw money and that the full banking system works.

That's a far more descriptive piece of content to an LLM. Being encouraged to write *for the LLM itself* makes all the difference.

(As an aside, LLMs see candidate *answers* in their context before the user's question - so just treating RAG as question answering doesn't work).

What often matters isn't algorithms. It's programming the content author to create useful LLM context. That moves content discovery from random, disconnected factoids to documentation connecting information to problems to solve.

You learn that the trick to RAG isn't creating good question answering systems. It's writing better answers.

The enterprise search morass... is over?

Compare how coding agents program us to the eternal nightmare of your company's wiki.

When people write in the wiki, they do it for themselves. Nobody is incentivized to make information findable by anyone. No author sits obsessing how to 'SEO' the content for Confluence search.

If someone is 'lucky' enough to find your article, they can barely understand it. It's written in your team's jargon. And it's probably out of date anyways.

For these reasons, I always tell people they'd do better to write a public blog article. Blogs assume zero context. They're clearly timestamped as an artifact of a moment in time. To avoid shame or promote your career, you're incentivized to create something useful, interesting, and findable.

Writing for a coding agent has the same property. You're writing documentation for a targeted reader. You care to SEO your content for Claude Code + grep.

Enterprise search fails because there's no feedback loop between authorship and findability. Coding agents solve that, finally giving some hope to organizing company information into findable knowledge.

The real outcome of coding agents isn't just code generation. It's closing the feedback loop between company context and the next piece of work being done. That old fable of Enterprise Search - improving workforce productivity by leveraging existing insight - can finally become true.

Feedback to content authors drives user relevance

Look around your system. Who creates the content? What's in it for them to make content findable?

A recurring theme in my consulting looks something like this - a job search company notices spammy behavior from job posters. They keyword stuff every programming language in some hidden text element. The usual reaction: that's spammy and we should ban them.

That's too black and white. You have content creators willing to put in work. Shift their black-hat SEO over to an acceptable white-hat SEO. Document what's acceptable. What's considered malicious.

Focus on feedback. Make the content-optimization path rewarding. Give creators feedback+guidance during creation. LLMs help detect spam in real-time. They push content creators to author what's useful not malicious.

Give content creators feedback. Once you've identified what 'good' is, give those content authors dopamine hits. Tell content creators what queries perform well for the content. Make it real-time, sticky, fun. Incentivize and reward behaviors you want - and punish those you don't.

Steer content creators towards the content you want. Don't hope algorithms can overcome drivel and slop.

Your search engine isn't a bundle of algorithms ranking for users. It's a relationship between authors and users. That bitter lesson puts grep above the smartest reranker and embedding model. Forget it at your peril.

Enjoy softwareDoug in training course form!

Starting May 18!

Signup here - <http://maven.com/softwareDoug/cheat-at-search>



I hope you join me at [Cheat at Search with Agents](#) to learn use agents in search, build better RAG and use LLMs in query understanding.

© 2026 SoftwareDoug LLC